# Лабораторная работа № 1 – Изучение методов защиты программного обеспечения

## Цель работы

Изучение методов защиты программного обеспечения и возможных способов их преодоления.

## Формулировка задания

В ходе выполнения лабораторной работы необходимо выполнить следующие действия:

1. Изучить подходы, применяемые для защиты программного обеспечения [1-22].
2. Реализовать программу (crackme) на компилируемом языке (C\C++, Go, Rust, Swift), осуществляющую считывание пароля из файла “password.txt” и в случае совпадения его с заданным, вывод сформированного серийного номера в формате KEY$xxxxxxxxxx$ (x – произвольные символы) в файл “serial.txt”, а также вывод сообщения об успехе в интерфейсе программы, в обратном случае - сообщения об ошибке. Все файлы должны быть расположены в одной директории с исполняемым файлом программы.
3. В качестве защищаемого программного обеспечения (ПО) может быть выбрана программа, ранее разработанная студентом, либо любое ПО с открытым исходным кодом.
4. Разрабатываемая вами программа будет **частью** задания из лабораторной работы №2, в которой необходимо будет решить crackme одногруппника. Соответственно, вам необходимо выбрать напарника (либо получить распределение у преподавателя), с которым нужно будет обменяться исполняемыми файлами (без предоставления исходного кода) разработанной программы для возможности дальнейшего их анализа и решения (в рамках Л.Р. №2).
5. В программе должен быть «полезный функционал», выполнение которого не должно происходить при введении неправильного пароля.

6. Осуществить модификацию исполняемого файла (бинарный патчинг) таким образом, чтобы результат проверки пароля всегда был положительным и генерировался верный серийный номер.

7. В отчете привести описание осуществленных действий для нахождения места проверки пароля и модификации файла.

8. Необходимо реализовать защиту разработанного программного обеспечения от возможности изменения поведения программы при помощи бинарного патчинга.

9. Реализованная программа должна использовать следующие общие подходы к защите ПО:

a. Хранение в зашифрованном виде всех строк, в т.ч. выводимых на экран.

b. Проверка и вывод информации в разных местах программы.

c. Контроль целостности участков кода, ответственных за проверку «пароля» (подсчет CRC исполняемого кода функции проверки пароля **в оперативной памяти** во время выполнения программы).

d. Наличие нескольких проверок пароля в разных местах программы.

e. Наличие ложных проверок (в т.ч. сразу после считывания «пароля»).

f. Использование методов запутывания кода для усложнения анализа кода программы.

10. Защитные механизмы должны быть тесно переплетены с логикой защищаемого кода и распределены по всему «полезному» коду, в случае модификации кода защитных механизмов код полезной нагрузки должен стать некорректным.

11. Реализованная программа должна использовать:

a. Не менее 4 методов обнаружения средств отладки и противодействия им.

b. Не менее 3 методов противодействия дизассемблированию (методы, которые приводят к некорректному дизассемблированию кода, см. главу 15 [1]).

c. Не менее 3 методов выявления виртуальных машин.

d. Фрагменты самомодифицирующегося кода.

e. Методы бинарной обфускации и обфускации потока передачи управления (нелинейная передача управления, например, через исключения [2].

12. Для реализованной программы произвести обход внедренных механизмов защиты путем бинарного патчинга исполняемого кода.

13. Произвести упаковку исполняемого файла с использованием одного из распространенных упаковщиков исполняемых файлов (например, UPX).

14. Провести анализ принципов работы используемого средства защиты бинарного кода с помощью дизассемблера, отладчика и других

программных средств. Найти в отладчике оригинальную точку входа в программу после упаковки файла.

15. Сравнить параметры оригинального и упакованного файлов (энтропия).

16. В отчете необходимо привести следующую информацию:

— исходный код программы до внесения в нее механизмов защиты;

— описание полезной функциональности программы;

— описание процесса модификации программы без механизмов защиты;

— перечень механизмов защиты и фрагменты исходного кода, отвечающие за их реализацию;

— описание принципа действия используемых механизмов защиты;

— исходный код программы после добавления механизмов защиты;

— описание процесса обхода реализованных механизмов защиты;

— описание программы-упаковщика;

— описание алгоритма упаковки исполняемого файла;

— фрагмент кода упакованной программы, отвечающий за самораспаковку исполняемого файла (загрузчик), полученный с помощью дизассемблера;

— схема структуры упакованного файла;

— параметры исполняемых файлов до и после упаковки (размер файла, энтропия и т.п.).

## Список источников


1. Michael Sikorski, Andrew Honig. Practical Malware Analysis
https://doc.lagout.org/security/Malware%20%26%20Forensics/Practical%20Malware%20Analysis.pdf
https://library.samdu.uz/files/1b7fcd1ad7f0fd6f3ac8a3ffe87166ec_Prakticheskiy_analiz_vrednosnogo_bzd_channel.pdf
2. https://anti-debug.checkpoint.com/
3. Binary Deobfuscation. https://calwa.re/reversing/obfuscation/binary-deobfuscation-preface#indirect-branches
4. Анти-дизассемблерные приемы
https://www.malwinator.com/2015/11/22/anti-disassembly-used-in-malware-a-primer/
https://www.malwinator.com/2015/11/27/anti-disassembly-techniques-used-by-malware-a-primer-part-2/
5. ANTI-DISASSEMBLY
http://staff.ustc.edu.cn/~bjhua/courses/security/2014/readings/anti-disas.pdf
6. Примеры используемых антиотладочных приемов в ВПО
https://github.com/rrbranco/blackhat2012/blob/master/blackhat2012-paper.pdf
https://github.com/rrbranco/blackhat2012
7. Introduction Into Windows Anti-Debugging
http://www.codeproject.com/Articles/29469/Introduction-Into-Windows-Anti-Debugging
8. An Anti-Reverse Engineering Guide
http://www.codeproject.com/Articles/30815/An-Anti-Reverse-Engineering-Guide
9. Anti-Debug Protection Techniques: Implementation and Neutralization
http://www.codeproject.com/Articles/1090943/Anti-Debug-Protection-Techniques-Implementation-an
10. The “Ultimate”Anti – Debugging Reference
http://pferrie.host22.com/papers/antidebug.pdf
11. OpenRCE Anti Reverse Engineering Techniques Database
http://www.openrce.org/reference_library/anti_reversing
12. Anti Reverse Engineering Protection Techniques to Use Before Releasing Software
https://www.apriorit.com/dev-blog/367-anti-reverse-engineering-protection-techniques-to-use-before-releasing-software
13. The art of unpacking
https://www.blackhat.com/presentations/bh-usa-07/Yason/Whitepaper/bh-usa-07-yason-WP.pdf

14. Anti-Memory Dumping Techniques
http://resources.infosecinstitute.com/anti-memory-dumping-techniques/
15. Пишем упаковщик PE-файлов по шагам
https://kaimi.ru/2012/09/pe-packer-step-by-step-1/
16. Об упаковщиках в последний раз: Часть первая - теоретическая
http://uinc.ru/articles/41/
17. Об упаковщиках в последний раз. Часть вторая
https://exelab.ru/art/?action=view&id=205
18. Разработка своего PE-упаковщика
https://habrahabr.ru/company/xakep/blog/139138/
19. Empirical identification, comparison and circumvention of current Antivirus detection techniques
https://www.blackhat.com/docs/us-14/materials/us-14-Mesbahi-One-Packer-To-Rule-Them-All-WP.pdf
20. Strings Obfuscation System
http://www.codeproject.com/Articles/502283/Strings-Obfuscation-System
21. Binary Obfuscation
https://www.defcon.org/images/defcon-17/dc-17-presentations/defcon-17-sean_taylor-binary_obfuscation.pdf
22. Binary code obfuscation through C++ template metaprogramming
https://www.scribd.com/document/328917840/2012-snfa2-pdf